

Banking Scorecard Data Model

Bridge_Account_Resp — Daily SCD Type 2 Update

Stored procedure to compare yesterday vs today and expire changed rows

1. Overview

Bridge_Account_Resp experiences frequent daily changes as accounts are reassigned between responsibility entities, credit percentages are adjusted, and new accounts are opened. The SCD Type 2 pattern preserves the full history of these assignments so that historical snapshots in FactGoalSnap always reflect the correct responsibility attribution for any given date.

The update process runs each night after the source system data has been loaded into a staging table. It compares today's source state against the current active rows in Bridge_Account_Resp, expires anything that has changed or been removed, and inserts new versions for every change.

2. Nightly Run Order

The following steps must run in sequence each night:

Step	Job	Dependency
1	Truncate Stage_Bridge_Account_Resp	Run before source load
2	Load Stage_Bridge_Account_Resp from source	Source system feed must be complete
3	EXEC usp_Update_Bridge_Account_Resp	Staging table must be loaded
4	EXEC usp_Load_FactGoalSnap_* procedures	Bridge must be updated first

Note: Bridge_Account_Resp must always be updated before the FactGoalSnap procedures run. If the bridge is stale the snapshot will attribute actuals to the wrong responsibility entity.

3. Staging Table

The staging table holds today's complete snapshot from the source system. It is truncated and reloaded every night before the SCD procedure runs. No SCD columns are needed — it only holds the current state.

```
CREATE TABLE Stage_Bridge_Account_Resp (  
  Account_Key INT NOT NULL,  
  Resp_Code VARCHAR(20) NOT NULL,  
  Credit_Pct DECIMAL(5,4) NOT NULL,  
  Credit_Reason_Cd VARCHAR(20) NOT NULL  
);
```

4. The Three Scenarios

Every row in either the bridge or the staging table falls into one of three scenarios. The procedure handles all three in two steps — expire then insert.

Scenario	In Bridge Today?	In Staging Today?	Step 1 — Expire?	Step 2 — Insert?
Unchanged	Yes — identical	Yes — identical	No	No — already exists
Changed	Yes — different	Yes — different	Yes	Yes — new version
Removed	Yes	No	Yes	No
New	No	Yes	No	Yes — new row

5. Stored Procedure — usp_Update_Bridge_Account_Resp

SQL Server — usp_Update_Bridge_Account_Resp

```

CREATE PROCEDURE usp_Update_Bridge_Account_Resp
@Process_Date DATE
AS
BEGIN
SET NOCOUNT ON;

-- ■■ Step 1: expire rows that have changed or disappeared ■■
-- A row needs expiring if:
-- A) The account still exists in source but something changed
-- B) The account no longer exists in source (removed assignment)

UPDATE bar
SET bar.Expiration_Date = DATEADD(DAY, -1, @Process_Date),
bar.Is_Current_Flag = 'N'
FROM Bridge_Account_Resp bar
LEFT JOIN Stage_Bridge_Account_Resp stg
ON stg.Account_Key = bar.Account_Key
AND stg.Resp_Code = bar.Resp_Code
AND stg.Credit_Pct = bar.Credit_Pct
AND stg.Credit_Reason_Cd = bar.Credit_Reason_Cd
WHERE bar.Is_Current_Flag = 'Y'
AND bar.Expiration_Date IS NULL
AND stg.Account_Key IS NULL; -- not in today source = changed or removed

-- ■■ Step 2: insert new versions for changed or new rows ■■■
-- A row needs inserting if:
-- A) It exists in source but not in current active bridge (new)
-- B) It exists in source but current active row was just expired (changed)

INSERT INTO Bridge_Account_Resp (
Account_Key,

```

[illegible]

6. Key Design Points

Expiration date is the day before

When a row is expired the `Expiration_Date` is set to `DATEADD(DAY, -1, @Process_Date)` — the day before the new version takes effect. This ensures there is no overlap between the old and new versions. A past date query for any date between `Effective_Date` and `Expiration_Date` inclusive will always resolve to exactly one row.

LEFT JOIN with IS NULL pattern

Both the expire step and the insert step use a LEFT JOIN with a WHERE IS NULL filter. This is the most efficient way in SQL Server to find rows that exist in one set but not the other. It avoids subqueries and performs well on large staging tables.

Comparison key includes all meaningful columns

The join between the bridge and staging uses `Account_Key`, `Resp_Code`, `Credit_Pct`, and `Credit_Reason_Cd`. If any of these four values change for an account the old row is expired and a new version is inserted. This captures every meaningful business change.

Past date reruns

If you need to rerun for a past date the procedure is not automatically idempotent — running it twice for the same date could create duplicate rows. Add a cleanup step before rerunning for a specific past date:

```
-- Cleanup before rerunning for a past date
-- Step 1: delete rows inserted on that date
DELETE FROM Bridge_Account_Resp
WHERE Effective_Date = @Process_Date;

-- Step 2: unexpire rows that were expired on that date
UPDATE Bridge_Account_Resp
SET Expiration_Date = NULL,
    Is_Current_Flag = 'Y'
WHERE Expiration_Date = DATEADD(DAY, -1, @Process_Date);

-- Step 3: now rerun the procedure
EXEC usp_Update_Bridge_Account_Resp @Process_Date = '2026-05-07';
```